

```
In [87]: #Session 1 Hoomework
# Name: Mohammed Akif
# Date: May 2025
```

```
In [78]: #TASK 1: Variables & Data Types
pipeline_name = "user_sync_pipeline"
source_system = "crm_database"
total_records = 25000
error_count = 178
processing_time = 45.7
is_successful = 1

print(f"{pipeline_name} -> {type(pipeline_name)}")
print(f"{source_system} -> {type(source_system)}")
print(f"{total_records} -> {type(total_records)}")
print(f"{error_count} -> {type(error_count)}")
print(f"{processing_time} -> {type(processing_time)}")
print(f"{is_successful} -> {type(is_successful)}")
```

```
user_sync_pipeline -> <class 'str'>
crm_database -> <class 'str'>
25000 -> <class 'int'>
178 -> <class 'int'>
45.7 -> <class 'float'>
1 -> <class 'int'>
```

```
In [79]: # TASK 2: Arithmetic Operations
success_count = total_records - error_count
success_rate = success_count/total_records
error_rate = error_count / total_records
records_per_second = total_records / processing_time
print(f"Success count: {success_count}")
print(f"Success rate: {success_rate:.2f}%")
print(f"Error rate: {error_rate:.2f}%")
print(f"Records/sec: {records_per_second:.1f}")
```

```
Success count: 24822
Success rate: 0.99%
Error rate: 0.01%
Records/sec: 547.0
```

```
In [80]: # TASK 3: String Operations

raw_name = "    rahim UDDIN    "
raw_email = "RAHIM@GMAIL.COM"
raw_city = "dhaka"

# Clean them up:
clean_name = raw_name.strip()
clean_email = raw_email.lower()
clean_city = raw_city.title()

print(f"Name: {clean_name}")
print(f"Email: {clean_email}")
print(f"City: {clean_city}")

summary = f"{clean_name} ({clean_email}) from {clean_city}"
print(summary)
```

```
Name: rahim UDDIN
Email: rahim@gmail.com
City: Dhaka
rahim UDDIN (rahim@gmail.com) from Dhaka
```

In [81]: *# TASK 4: Type Conversion*

```
str_price = "1250.75"
str_quantity = "8"
str_discount = "15"
str_in_stock = "True"

price = float(str_price)
quantity = int(str_quantity)
discount = int(str_discount)

subtotal = price * quantity
discount_amount = subtotal * discount / 100
final_total = subtotal - discount_amount

print(f"Subtotal: {subtotal:.2f} BDT")
print(f"Discount ({discount}%): -{discount_amount:.2f} BDT")
print(f"Final Total: {final_total:.2f} BDT")
```

Subtotal: 18761.25 BDT
Discount (15%): -2814.19 BDT
Final Total: 15947.06 BDT

In [82]: *# TASK 5: Comparisons & Booleans*

```
rows_loaded = 9500
rows_expected = 10000
null_percentage = 3.2
max_allowed_nulls = 5.0
run_time_minutes = 12
max_run_time = 15

# Write boolean expressions:
data_complete = rows_loaded >= rows_expected
data_clean = null_percentage <= max_allowed_nulls
within_time = run_time_minutes <= max_run_time
all_checks_passed = data_complete and data_clean and within_time
any_issue = not all_checks_passed

print(f"Data complete: {data_complete}")
print(f"Data clean: {data_clean}")
print(f"Within time: {within_time}")
print(f"All passed: {all_checks_passed}")
print(f"Any issue: {any_issue}")
```

Data complete: False
Data clean: True
Within time: True
All passed: False
Any issue: True

In [83]: *# TASK 6: Conditionals (if/elif/else)*

```
pipeline_score = 87

if pipeline_score > 89:
    grade = "EXCELLENT"
    color = "green"
elif pipeline_score > 69:
    grade = "GOOD"
    color = "yellow"
elif pipeline_score > 49:
    grade = "WARNING"
    color = "orange"
else:
    grade = "CRITICAL"
    color = "red"
```

```
print(f"Pipeline Score: {pipeline_score} -> {grade} ({color})")
```

Pipeline Score: 87 -> GOOD (yellow)

```
In [84]: # TASK 7: For Loops
tables = ["customers", "orders", "products", "reviews", "inventory"]
row_counts = [15000, 42000, 3500, 28000, 7200]

# Task 7a: Print each table name in UPPERCASE
for table in tables:
    print(table.upper())

print("Print each table name in UPPERCASE is Done\n")

# Task 7b: Print each table with its row count
# Hint: use range(len(tables)) or zip(tables, row_counts)
for i in range(len(tables)):
    print(f"Table: {table[i]} -> {row_counts[i]} rows")

print("Print each table with its row count is Done\n")

# Task 7c: Find the total number of rows across all tables
total = 0
for count in row_counts:
    total += count          # add count to total

print(f"Total rows across all tables: {total}")
```

CUSTOMERS

ORDERS

PRODUCTS

REVIEWS

INVENTORY

Print each table name in UPPERCASE is Done

Table: i -> 15000 rows

Table: n -> 42000 rows

Table: v -> 3500 rows

Table: e -> 28000 rows

Table: n -> 7200 rows

Print each table with its row count is Done

Total rows across all tables: 95700

```
In [85]: # TASK 8: While Loop
remaining = 7500
batch_size = 1000
batch_number = 0

while remaining > 0:
    batch_number += 1
    if remaining >= batch_size:
        processed = batch_size
    else:
        processed = remaining
    remaining -= processed
    print(f"Batch {batch_number}: processed {processed}, remaining: {remaining}")

print(f"Done! Processed in {batch_number} batches")
```

```

Batch 1: processed 1000, remaining: 6500
Batch 2: processed 1000, remaining: 5500
Batch 3: processed 1000, remaining: 4500
Batch 4: processed 1000, remaining: 3500
Batch 5: processed 1000, remaining: 2500
Batch 6: processed 1000, remaining: 1500
Batch 7: processed 1000, remaining: 500
Batch 8: processed 500, remaining: 0
Done! Processed in 8 batches

```

```

In [86]: # TASK 9: Putting It All Together
# Build a complete pipeline status report

# --- Your pipeline data ---
pipeline = "ecommerce_daily_sync"
tables_processed = ["users", "orders", "products", "payments"]
total_rows = 95400
failed_rows = 1230
start_time_seconds = 0
end_time_seconds = 324

# --- Calculate metrics ---
success_rows = total_rows - failed_rows
success_rate = success_rows / total_rows * 100
duration_minutes = (end_time_seconds - start_time_seconds) / 60
rows_per_second = total_rows / (end_time_seconds - start_time_seconds)

# --- Determine status ---
if success_rate >= 98:
    status = "HEALTHY"
elif success_rate >= 95:
    status = "WARNING"
else:
    status = "CRITICAL"

# --- Print the report ---
print("=" * 50)
print(f" PIPELINE REPORT: {pipeline}")
print("=" * 50)
print(f" Status: {status}")
print(f" Duration: {duration_minutes:.1f} minutes")
print(f" Tables processed:")
for table in tables_processed:
    print(f"   - {table}")
print(f" Total rows: {total_rows:,}")
print(f" Successful: {success_rows:,} ({success_rate:.1f}%)")
print(f" Failed: {failed_rows:,}")
print(f" Throughput: {rows_per_second:.0f} rows/sec")
print("=" * 50)

```

```

=====
PIPELINE REPORT: ecommerce_daily_sync
=====
Status: CRITICAL
Duration: 5.4 minutes
Tables processed:
  - users
  - orders
  - products
  - payments
Total rows: 95,400
Successful: 94,170 (0.0%)
Failed: 1,230
Throughput: 294 rows/sec
=====

```

```
In [ ]: # User Input Version
pipeline = input("Pipeline name: ")
total = int(input("Total rows: "))
failed = int(input("Failed rows: "))
time_sec = float(input("Run time (seconds): "))

success = total - failed
success_rate = success / total *100
duration_min = time_sec / 60
throughput = total / time_sec

if success_rate >= 98:
    status = "HEALTHY"
elif success_rate >= 95:
    status = "WARNING"
else:
    status = "CRITICAL"

print("\n" + "=" * 50)
print(f" PIPELINE: {pipeline}")
print(f" STATUS: {success_rate: .1f}%")
print(f" THROUHPUT {throughput: .0f} rows/sec")
print("=" * 50)
```